

Горчак Дмитрий, 212. Библиотека STL: контейнеры, итераторы, алгоритмы. Двухнаправленный список и упорядочивание объектов

STL (Standard Template Library – библиотека стандартных шаблонов) – часть стандартной библиотеки C++, в которой лежат готовые структуры данных и алгоритмы: списки, массивы, словари, сортировка, поиск. Их не нужно писать самому – достаточно подставить свой тип данных.

В первой части разобрано, из чего состоит STL и какие в ней контейнеры. Во второй – как сложить объекты в двухнаправленный список и упорядочить его по разным признакам. Примеры взяты из учебного проекта cones: программа читает из файла параметры конусов (координаты центра, радиус основания, высоту) и считает у каждого площадь основания и объём; конусы нужно расположить сначала по возрастанию площади, потом по возрастанию объёма.

Из чего состоит STL

- 1) контейнеры (container – хранилище) – объекты, которые хранят другие объекты: vector, list, deque, set, map;
- 2) итераторы (iterator – перебирающий) – единый способ проходить содержимое любого контейнера;
- 3) алгоритмы – готовые операции: сортировка, поиск, подсчёт, копирование, преобразование;
- 4) функциональные объекты и лямбда-функции – ими задают правило сравнения или условие отбора и передают алгоритму;
- 5) адаптеры (stack, queue, priority_queue) и распределители памяти.

Шаблон (template – образец) означает, что тип хранимых данных не задан заранее, а подставляется при использовании: `std::list<int>` – список чисел, `std::list<Cone>` – список конусов. Код контейнера написан один раз и работает с любым типом.

Контейнеры ничего не знают об алгоритмах, а алгоритмы – о контейнерах; связывают их итераторы. Поэтому сортировка написана один раз и работает с разными контейнерами.

Контейнеры

Для каждой операции в стандарте указано, как растёт время её работы с увеличением числа элементов n : линейно (пропорционально n), за $\log n$ (почти не растёт) или за постоянное время. Выбор контейнера – это выбор того, какие операции окажутся дешёвыми.

Контейнер	Как устроен	Дёшево	Дорого
vector	динамический массив	доступ по номеру, добавление в конец	вставка и удаление в середине
list	двусвязный список	вставка и удаление в любом месте	нет доступа по номеру, поиск перебором
forward_list	односвязный список	минимум памяти на элемент	проход только вперёд
deque	блоки элементов	добавление с обоих концов	вставка в середину

set, map	сбалансированное дерево	поиск и вставка за $\log n$, лишняя память на связи порядок
unordered_set, map	хеш-таблица	поиск за постоянное время элементы не упорядочены

Адаптер (adapter – переходник) – не самостоятельное хранилище, а надстройка над контейнером, которая оставляет только нужные операции: stack (стек – кладём и снимаем с одного конца), queue (очередь – кладём в конец, берём из начала), priority_queue (первым выдаётся наибольший элемент).

Итераторы

Итератор – объект, который указывает на элемент контейнера и умеет переходить к соседнему. Его разыменовывают (*it – получить сам элемент), сдвигают (++it) и сравнивают с другим итератором.

Диапазон задаётся парой итераторов [first, last): last указывает уже за последний элемент, поэтому цикл заканчивается по условию `it != end()`, а не `it <= end()`.

Итераторы различаются тем, что умеют. У вектора итератор произвольного доступа (random access): к нему можно прибавить число и сразу попасть на нужный элемент. У списка итератор двунаправленный (bidirectional): он умеет шагать только вперёд и назад, по одному элементу. Отсюда следствие: общий алгоритм `std::sort` со списком работать не будет.

Алгоритмы

Готовые операции лежат в заголовочных файлах `<algorithm>` и `<numeric>`: `sort` и `stable_sort` (упорядочить), `find_if` (найти первый подходящий элемент), `count_if` (сосчитать подходящие), `max_element` и `min_element` (найти крайний по заданному правилу), `accumulate` (накопить сумму), `remove_if` (убрать подходящие).

Правило сравнения или отбора передаётся отдельным аргументом, поэтому одна и та же сортировка упорядочивает и по площади, и по объёму.

Двунаправленный список: устройство

Двунаправленный (двусвязный) список состоит из заголовка и элементов. Заголовок хранит указатель на первый элемент и число элементов `n`. Каждый элемент хранит три вещи: сам объект, ссылку на предыдущий элемент и ссылку на следующий.

[заголовок: $n = 3$, first] --> [Cone A] <--> [Cone B] <--> [Cone C]

Из такого устройства следуют все свойства списка:

- 1) по списку можно идти в обе стороны – от начала и от конца;
- 2) вставка и удаление сводятся к переключению двух ссылок: соседние элементы не сдвигаются и не копируются;
- 3) обращения по номеру нет – чтобы добраться до пятого элемента, нужно пройти через четыре предыдущих;
- 4) на каждый элемент тратится дополнительная память под две ссылки.

В стандартном `std::list` список замкнут в кольцо через служебный элемент, а число элементов хранится в заголовке, поэтому `size()` выполняется мгновенно. Адреса объектов при вставках, удалениях и сортировке не меняются – переставляются только ссылки.

Создание списка и добавление объектов

Нужен заголовочный файл `<list>`. Список объявляется одной строкой, а объекты добавляются по мере создания:

```
#include <list>

std::list<Cone> cones;

for (std::size_t i = 0; i < loaded.size(); ++i) {
    cones.push_back(loaded[i]);    /* новый элемент в конец списка */
}
```

`push_back` добавляет в конец, `push_front` – в начало, `front` и `back` возвращают первый и последний элементы, `size` – число элементов, `empty` – пуст ли список.

Перебор идёт итератором; записи вида `cones[3]` у списка нет:

```
for (std::list<Cone>::const_iterator it = cones.begin(); it != cones.end(); ++it) {
    std::cout << it->get_r() << ' ' << it->area() << ' ' << it->volume() << '\n';
}
```

`const_iterator` – итератор, через который элементы можно только читать. Стрелка `it->` обращается к методу того объекта, на который итератор сейчас указывает.

Упорядочивание по площади и по объёму

Порядок задаётся функцией сравнения: она получает два объекта и возвращает `true`, если первый должен стоять раньше. Площадь и объём у конуса не хранятся в полях, а считаются методами, поэтому сравнение обращается к методам:

```
bool less_by_area(const Cone &left, const Cone &right) {
    return left.area() < right.area();
}

bool less_by_volume(const Cone &left, const Cone &right) {
    return left.volume() < right.volume();
}

cones.sort(less_by_area);    /* порядок по площади основания */
cones.sort(less_by_volume); /* порядок по объёму */
```

Вызывается не общий алгоритм `std::sort`, а собственный метод списка `sort`. Причина в итераторах: `std::sort` прыгает по элементам туда-сюда, а итератор списка так не умеет. Метод `list::sort` реализован сортировкой слиянием: он переставляет ссылки между элементами, не копируя сами объекты, работает за $n \log n$ и сохраняет взаимный порядок равных элементов (такую сортировку называют устойчивой).

Если список уже упорядочен, новый элемент можно сразу поставить на своё место. Поиск места – проход по списку, сама вставка стоит двух

переключений ссылок независимо от длины списка:

```
void insert_sorted_by_volume(std::list<Cone> &cones, const Cone &value) {
    std::list<Cone>::iterator place = cones.begin();

    while (place != cones.end() && place->volume() < value.volume()) {
        ++place;
    }
    cones.insert(place, value);    /* insert ставит элемент перед place */
}
```

Удаление по условию делается одной командой: `remove_if` проходит список и выбрасывает подходящие элементы, не трогая остальные.

```
cones.remove_if(small_volume);    /* убрать конусы с объёмом меньше порога */
```

Результат

Два порядка получаются разными. Заметнее всего это на конусе с широким, но низким основанием ($r = 2.80$, $h = 2.00$): по площади он предпоследний, а по объёму – только седьмой. Столбцы вывода: номер, x , y , радиус, высота, площадь основания, объём.

--- Sorted by base area ---

9	19.00	4.00	2.20	9.00	15.21	45.62
10	3.00	5.50	2.80	2.00	24.63	16.42
11	9.00	8.00	3.00	10.00	28.27	94.25

--- Sorted by volume ---

7	3.00	5.50	2.80	2.00	24.63	16.42
8	15.00	15.50	1.80	6.50	10.18	22.05
9	19.00	4.00	2.20	9.00	15.21	45.62

Двунаправленность проверяется обратным проходом: тот же список читается от конца к началу обратным итератором (`rbegin` и `rend`), и объёмы идут по убыванию.

```
for (std::list<Cone>::const_reverse_iterator it = cones.rbegin();
     it != cones.rend(); ++it) {
    std::cout << it->volume() << ' ';
}
```

Список или вектор

Список выгоден, когда элементы добавляются и удаляются по одному, а доступ к ним последовательный: объекты создаются при чтении файла и потом упорядочиваются целиком. Вектор выгоден, когда нужен доступ по номеру и важна скорость перебора: его элементы лежат в памяти подряд, и процессор читает их быстрее.

Часто в одной задаче нужны оба: например, матрица попарных расстояний n на n – это вектор векторов с доступом по номерам, а очередь точек при обходе графа – адаптер `queue`.

Заключение

Готовый шаблон списка избавляет от написания своей структуры данных: достаточно подставить нужный тип и задать правила сравнения. Понимать устройство всё равно нужно – именно из него следует, почему у списка своя

сортировка, почему нет обращения по номеру и почему вставка в середину дешевле, чем в векторе.

*[объект создан] -(push_back)-> [элемент списка: объект + две ссылки]
 [правило сравнения] -(list::sort)-> [порядок по площади или по объёму]
 [итератор] -(++ / --)-> [проход вперёд и назад]*

Приложение. Памятка по `std::list`

Цель	Операция	Пример	Описание
Объявить список	<code>std::list<T></code>	<code>std::list<Cone> cones;</code>	T – тип хранимых объектов
Добавить в конец	<code>push_back</code>	<code>cones.push_back(cone);</code>	новый элемент становится последним
Добавить в начало	<code>push_front</code>	<code>cones.push_front(cone);</code>	новый элемент становится первым
Вставить в середину	<code>insert</code>	<code>cones.insert(place, cone);</code>	ставит элемент перед итератором <code>place</code>
Удалить элемент	<code>erase</code>	<code>cones.erase(place);</code>	убирает элемент, на который указывает <code>place</code>
Удалить по условию	<code>remove_if</code>	<code>cones.remove_if(small_volume);</code>	убирает все подходящие элементы
Первый и последний	<code>front, back</code>	<code>cones.front().volume();</code>	возвращают сам объект, а не итератор
Число элементов	<code>size, empty</code>	<code>cones.size();</code>	<code>size</code> выполняется мгновенно
Проход вперёд	<code>begin, end</code>	<code>for (it = cones.begin(); ...)</code>	<code>end</code> указывает за последний элемент
Проход назад	<code>rbegin, rend</code>	<code>for (it = cones.rbegin(); ...)</code>	обратный итератор, от конца к началу
Упорядочить	<code>sort</code>	<code>cones.sort(less_by_area);</code>	сортировка слиянием, $n \log n$, устойчивая
Перевернуть	<code>reverse</code>	<code>cones.reverse();</code>	меняет порядок элементов на обратный
Объединить списки	<code>merge, splice</code>	<code>a.merge(b);</code>	переносит элементы без копирования
Очистить	<code>clear</code>	<code>cones.clear();</code>	удаляет все элементы